EL PERRI LATIN FOOD

Website Review & Feature Report

A full walkthrough of everything the site does, what was tested, what was fixed, and what to handle before launch.

Live site	**elperrilatinfood.vercel.app**
Prepared for	**The El Perri team**
Date	**June 25, 2026**
Scope	**Every public & admin feature, code review, and live testing**

Bottom line: the site is live and looks great, and the core features really work. Before you collect real customer data, a few security items need attention — all listed in Section 6, with the main one already prepared as a fix.

Executive summary

El Perri's website is a polished, modern, fully bilingual restaurant site with a live, database-driven menu, an interactive “what should I order?” assistant, a daily-lunch email program, and a working admin panel the owner can run without a developer. Visually it is strong: a video/photo hero, brand story, photo gallery, an autoplaying story video, and a live Instagram feed.

Everything customer-facing was tested live and works. The admin panel’s core tools (menu, daily lunch, subscribers, users) are real and update the site instantly. The main caveats are not about whether things work — they do — but about security hardening and email delivery that should be addressed before the site is promoted and starts collecting real customer data.

AT A GLANCE	
Site status	LIVE
Public features tested	**8 / 8 working**
Admin tools (real, working)	**Menu, Daily lunch, Subscribers, Users, Stats**
Issues fixed today	**7 (security cleanup + bug fixes), committed to a branch**
Before launch	**3 critical, 2 important items (Section 6)**

1 What’s on the website (customer-facing)

Every page and feature below was opened and exercised on the live site.

Feature	What it does	Status
Homepage	Full-screen food hero, brand story, scrolling specialty banner, featured dishes, photo gallery, autoplay story video, and a live Instagram feed.	WORKING
Menu page	The complete menu (63 items, grouped by category) loaded live from the database. Admin edits appear here instantly.	WORKING
“¿Qué pido?” assistant	Friendly chatbot that asks a few taste questions and recommends dishes, with a favorites heart. Runs in the browser — no AI cost.	WORKING
Daily-lunch sign-up	Pop-up “¿Quieres saber el almuerzo de hoy?” captures name + email straight into the live database.	WORKING
Nuestra Historia	Brand story page — “De Colombia pal mundo.”	WORKING
Catering	Catering pitch and what’s offered for events.	WORKING
Account create / login	Visitors can make an account or sign in from the welcome pop-up. Works, but see security note in Section 6.	WORKING*

Feature	What it does	Status
Mobile / responsive	Layout adapts to phones and tablets, with a collapsing mobile menu.	WORKING
Online ordering	Shown as “Próximamente” (coming soon) by design. Checkout is a placeholder, not yet active.	BY DESIGN

2 The admin panel

Reached at /admin/login. After sign-in, the owner gets a dashboard of tabs. The tools below marked “live” read and write the real database; the “demo” tabs are placeholders not yet connected to real data.

Feature	What it does	Status
Menu management	Add, edit, delete dishes. Changes sync to the public menu in real time (“EN VIVO”).	LIVE
Almuerzo del día	Set today’s lunch, see subscribers, send the email now, and export subscribers to CSV.	LIVE
Usuarios y Marketing	List of registered users, with copy-emails and CSV export for marketing.	LIVE
Estadísticas	Counts (e.g. 63 dishes on the menu). Orders/revenue/staff read 0 — honest, since those aren’t wired up yet.	LIVE
Pedidos, Clientes, Inventario, Personal, Promociones, Ajustes	Present as tabs but store data only in the local browser — placeholders for future build-out, not real shared data.	DEMO

Recommendation: hide the demo tabs until they’re wired to real data, so the panel only shows tools that actually do something. (Listed again in Section 6.)

3 How the daily-lunch email works

This is the feature set up most recently, so here’s the full chain in plain terms:

1. Customer subscribes
pop-up on site
→
2. Saved to database
Supabase
→
3. Sent daily / on demand
Vercel Cron 16:00 UTC
or “Enviar correo ahora”
→
4. Delivered by email
Resend

It is confirmed working: today there is 1 subscriber (the owner’s own address) and sending succeeds. **Important:** email currently goes out from Resend’s shared test address, which only delivers to the account owner. To reach real customers you must verify your own domain in Resend and set the EMAIL_FROM value — see Section 6.

4 Testing results

Test	Result	Outcome
Homepage + all sections	Loaded; hero, gallery, story video autoplay, live Instagram feed all render	PASS
Menu (live database)	63 items load and group correctly	PASS
Order assistant flow	Completed a full flow; returned sensible dish recommendations	PASS
Subscribe pop-up	Captures to database; subscriber appears in admin	PASS
Nuestra Historia / Catering	Both load and render correctly	PASS
Admin login + dashboard	Sign-in works; tabs load	PASS
Admin: daily lunch	Special saved; subscriber list + CSV export present	PASS
Admin: menu / users / stats	Live data reads correctly	PASS
Responsive / mobile	Breakpoints + mobile menu present in code	PASS
Production build	Could not run in our sandbox (no network for Next's Linux binary); Vercel builds it on deploy	N/A

5 What I fixed today

These are done and committed to a branch named **fix/security-and-cleanup** (kept off the live **main** branch per your project rules, ready for you to deploy):

Fix	Why it mattered
Removed an open customer-data export	An admin "export users" web address had **no password check** — anyone could have called it. Deleted.
Removed 6 more unused/insecure back-end routes	Leftover scaffolding that either had no security or returned fake numbers. Deleted to shrink the attack surface.
Removed a fake-metrics screen	An unused dashboard component showed invented figures ($1,847 revenue, fake orders). Deleted so nothing fabricated can ever show.
Fixed a broken link in the daily email	The "Ver el menú" button pointed to the wrong web address; now points to your real site.

Fix	Why it mattered
Removed a 32 MB program from the code	A networking tool (ngrok.exe) was committed into the project. Removed and blocked from returning.
Added a database security fix file	A ready-to-apply security policy file (with instructions) for the critical item in Section 6.
Fixed broken maintenance scripts	Database helper commands pointed at files that don’t exist; corrected.

6 Before you go live: priorities

None of these stop the site from working today, but the first three should be handled before the site is promoted and starts collecting real customer information.

CRITICAL **Lock down the database**

Right now the database's access rules are wide open: the public site key (visible in any browser) can read **every subscriber and customer email** and account record, and can change menu/orders/promotions. This must be tightened before real customer data is collected. I've added the exact fix as **db/supabase-harden-rls.sql**; it needs a small paired developer change (use a private server key for the admin/email) so the panel keeps working — steps are in the file.

CRITICAL **Stop storing passwords in plain text**

The "create account" feature saves passwords unencrypted and checks them in the browser. Recommend switching account sign-in to a proper system (Supabase Auth), or turning off account creation until then. (The daily-lunch email sign-up does **not** use passwords and is fine.)

CRITICAL **Set a strong admin username + PIN**

Unless you've already set ADMIN_USERNAME and ADMIN_PIN in Vercel, the panel falls back to the defaults **admin / 1234**. Set a strong username and PIN in Vercel → Settings → Environment Variables, then redeploy.

IMPORTANT **Make the daily email reach real customers**

Email is sent from Resend's shared test address, which only delivers to your own inbox — that's why "1 subscriber" works but real customers wouldn't receive it. Verify your domain in Resend and set EMAIL_FROM (e.g. "El Perri <hola@elperrilatinfood.com>").

IMPORTANT **Decide on online ordering**

Ordering shows "Próximamente." The checkout is a placeholder that doesn't take real orders. Either connect a real platform (DoorDash / Square / Toast) or finish the checkout before promoting ordering.

NICE TO HAVE **Polish items**

Remember when a visitor dismisses the sign-up pop-up so it doesn't reappear every visit; hide the admin "demo" tabs until built; and tidy the project (remove the unused Prisma dependency and the many auto-generated audit notes).

7 Technical appendix (for your developer)

Architecture. Next.js (App Router) on Vercel; Supabase (Postgres) for menu, daily_special, subscribers, registered_users; Resend for email; a Vercel Cron hits /api/cron/daily-lunch daily. Public menu and admin both talk to Supabase from the browser with the anon key.

Root security issue. db/supabase-schema.sql enables RLS but then opens every table with `for all using (true) with check (true)`. Because the anon key ships to the browser, all tables are world read/write. registered_users.password is plain text. Fix = apply db/supabase-harden-rls.sql AND move admin/cron reads of PII to server routes using the service-role key (the file documents this; applying the SQL alone will break the current client-side admin/cron, so do both together).
Also worth doing. Admin session is a client-side sessionStorage flag only; lib/auth.js verifyAdminToken() was a mock that always returned valid (its routes are now deleted). The remaining auth/user-login, auth/user-signup, auth/logout, csrf-token and orders routes are unused and can be removed too. CSP in next.config.js is applied to /api/* (JSON) instead of pages.
Deploying these fixes. Branch `fix/security-and-cleanup` is committed locally (1 commit, 14 files: ngrok + 7 routes + 1 component removed, email link + package.json fixed, RLS SQL added). Push it, open a PR, run `npm run build` in CI, then merge to main — Vercel auto-deploys. The local repo also had pre-existing uncommitted edits to a few .md docs that were left untouched.

Prepared as a functional review of the El Perri website. “Working” means verified on the live site on June 25, 2026. Security findings reflect the code as deployed at that time.